INTRODUCTION

This report presents ten Python Activity Based Learning case studies. Each case is organized with its problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python program, test cases, output screenshot, and conclusion. The Python programs and output screenshots are compiled from the files supplied in the Python folder.

CONTENTS — TEN CASE STUDIES

Case 01. Advanced Numeric Pattern Generator
Case 02. Repeated Transaction Detection
Case 03. Password Policy Validation
Case 04. Student Email and Identifier Validation
Case 05. Student Marks Matrix Analysis
Case 06. Weekly Electricity Consumption Matrix
Case 07. Employee Shift Conflict Detection
Case 08. Product Catalogue Audit
Case 09. Environmental Monitoring Data Analyzer
Case 10. Advanced String-Based Registration Record Analyzer

Note: The Python code and output screenshots are taken from the submitted files. The problem statements, objectives, assumptions, algorithms, flowcharts, pseudocode, test cases and conclusions have been written from the logic of each program to complete the requested report structure.

CASE 01: Advanced Numeric Pattern Generator

1. Problem Statement

Pattern printing is a basic way to practise nested loops, spacing and conditional logic. This project develops a Python program that prints a hollow diamond pattern of stars for every value of n in a list of test cases. The upper half of the diamond grows from a single star to the widest row, and the lower half mirrors it back to a single star.

2. Objectives

1. To develop a Python program that generates a hollow diamond pattern for a given value of n.
2. To use nested loops to control leading spaces and the positions of the stars.
3. To print stars only at the first and last positions of each row.
4. To run several test values automatically using a list of test cases.
5. To display each pattern under a clear heading.

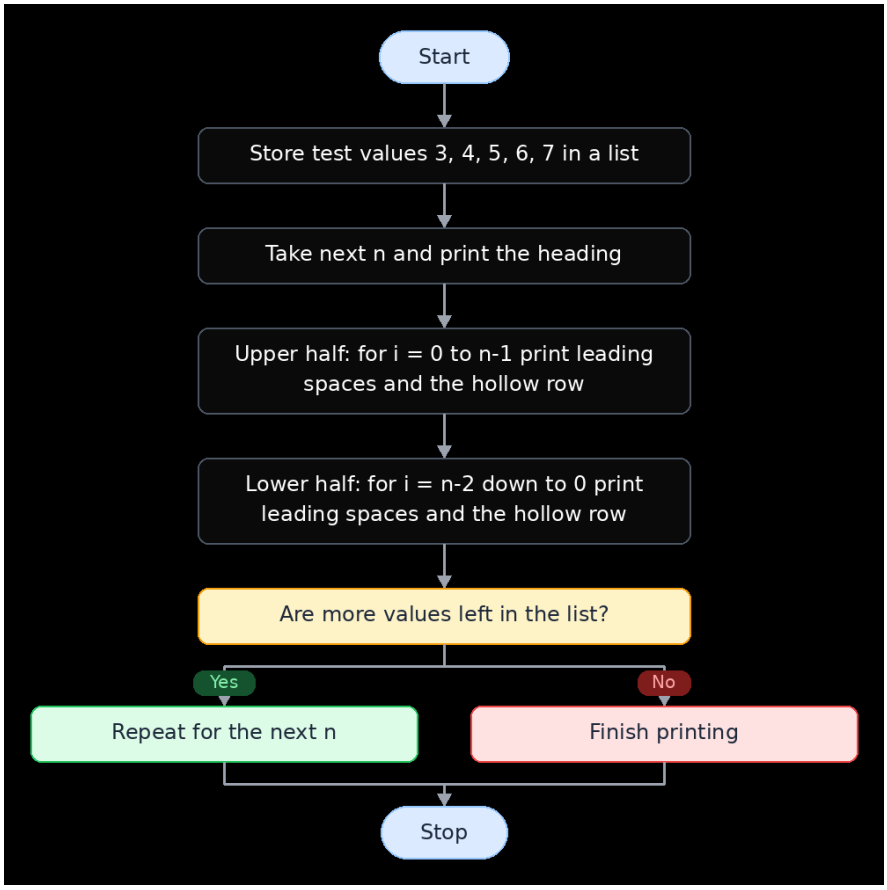
3. Assumptions

1. The value of n is a positive integer greater than 1.
2. The test values used are 3, 4, 5, 6 and 7.
3. The pattern contains 2n - 1 rows and its widest row contains 2n - 1 positions.
4. Row i of the upper half has n - i - 1 leading spaces and 2i + 1 positions.
5. A star is printed at position j = 0 and j = 2i; every other position is a space.
6. The lower half repeats the rows of the upper half in reverse order, excluding the widest row.

4. Algorithm

1. Start the program.
2. Store the test values 3, 4, 5, 6 and 7 in a list.
3. Take the next value of n from the list and print the heading Pattern for n.
4. For each row i from 0 to n - 1, print n - i - 1 leading spaces.
5. For each position j from 0 to 2i, print a star if j is 0 or 2i; otherwise print a space.
6. Move to the next line after each row to complete the upper half.
7. Repeat the same row logic for i from n - 2 down to 0 to print the lower half.
8. Repeat for the remaining values in the list.
9. Stop the program.

5. Flowchart



6. Pseudocode

```
START

SET test_cases = [3, 4, 5, 6, 7]

FOR each n in test_cases:
    DISPLAY "Pattern for n =", n

    FOR i = 0 TO n - 1:
        PRINT (n - i - 1) spaces
        FOR j = 0 TO 2*i:
            IF j = 0 OR j = 2*i:
                PRINT "*"
            ELSE:
                PRINT " "
        MOVE to next line

    FOR i = n - 2 DOWN TO 0:
        PRINT (n - i - 1) spaces
        FOR j = 0 TO 2*i:
            IF j = 0 OR j = 2*i:
                PRINT "*"
            ELSE:
                PRINT " "
        MOVE to next line

STOP
```

7. Python Source Code / Program

```
test_cases = [3, 4, 5, 6, 7]

for n in test_cases:
    print("\nPattern for n =", n)

    for i in range(n):
        print(" " * (n-i-1), end="")
        for j in range(2*i+1):
            if j == 0 or j == 2*i:
                print("*", end="")
            else:
                print(" ", end="")
        print()

    for i in range(n-2, -1, -1):
        print(" " * (n-i-1), end="")
        for j in range(2*i+1):
            if j == 0 or j == 2*i:
                print("*", end="")
            else:
                print(" ", end="")
        print()
```

8. Test Cases

Test	Input (n)	Expected result
1	3	Hollow diamond with 5 rows; widest row has 5 positions
2	4	Hollow diamond with 7 rows; widest row has 7 positions
3	5	Hollow diamond with 9 rows; widest row has 9 positions
4	6	Hollow diamond with 11 rows; widest row has 11 positions
5	7	Hollow diamond with 13 rows; widest row has 13 positions

9. Output Screenshot



10. Conclusion

This program prints a hollow diamond pattern for each test value using nested loops and simple conditions. It shows how spacing and position checks can be combined to control the shape of console output.

CASE 02: Repeated Transaction Detection

1. Problem Statement

Financial systems must detect repeated transaction IDs and highlight unusually large payments. Duplicate entries may indicate double billing or fraud, and large transactions often need extra verification. This project develops a Python program that checks several groups of transactions, lists any repeated transaction IDs and reports the transactions whose amount exceeds a limit.

2. Objectives

1. To develop a Python program that analyses groups of transactions.
2. To detect transaction IDs that appear more than once in a group.
3. To store the repeated IDs in a separate duplicates list.
4. To identify high-value transactions above the defined limit.
5. To display the transactions, duplicates and high transactions for every test case.

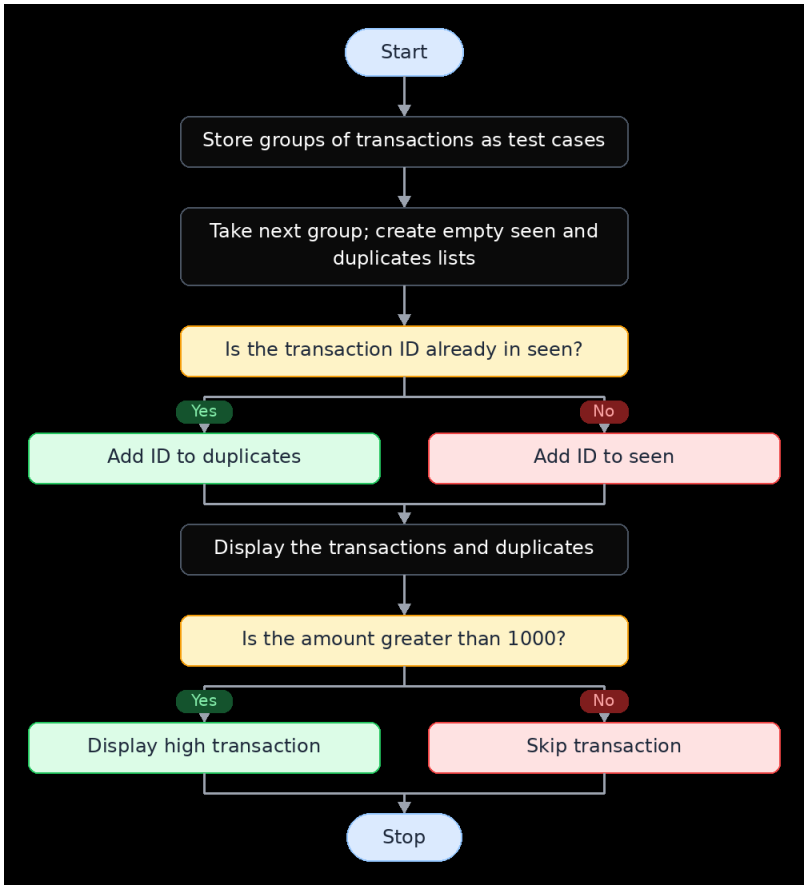
3. Assumptions

1. Each transaction is stored as a pair of transaction ID and amount.
2. A transaction is a duplicate if its ID has already appeared earlier in the same group.
3. A transaction is high-value if its amount is greater than 1000.
4. Every group of transactions is checked independently of the other groups.
5. Amounts are positive numbers.
6. Both entries of a repeated transaction are checked for the high-value limit.

4. Algorithm

1. Start the program.
2. Store the groups of transactions in a list of test cases.
3. For each group, create an empty seen list and an empty duplicates list.
4. For each transaction, check whether its ID is already in the seen list.
5. If it is present, add the ID to the duplicates list; otherwise add it to the seen list.
6. Display the transactions and the duplicates list.
7. Check every transaction in the group and display it as a high transaction if the amount is greater than 1000.
8. Repeat for all groups.
9. Stop the program.

5. Flowchart



6. Pseudocode

```
START

SET test_cases = groups of (transaction_id, amount)

FOR each group in test_cases:
    seen = empty list
    duplicates = empty list

    FOR each (tid, amount) in group:
        IF tid is in seen:
            ADD tid to duplicates
        ELSE:
            ADD tid to seen

    DISPLAY group
    DISPLAY duplicates

    FOR each (tid, amount) in group:
        IF amount > 1000:
            DISPLAY "High transaction:", tid, amount

STOP
```

7. Python Source Code / Program

```
test_cases = [
    [("T1", 500), ("T2", 1000), ("T1", 500)],
    [("T3", 200), ("T4", 300)],
    [("T5", 1500), ("T5", 1500)],
    [("T6", 700), ("T7", 2000), ("T7", 2000)],
    [("T8", 100), ("T9", 2500), ("T10", 400)]
]

for transactions in test_cases:
    seen = []
    duplicates = []

    for tid, amount in transactions:
        if tid in seen:
            duplicates.append(tid)
        else:
            seen.append(tid)

    print("\nTransactions:", transactions)
    print("Duplicates:", duplicates)

    for tid, amount in transactions:
        if amount > 1000:
            print("High transaction:", tid, amount)
```

8. Test Cases

Test	Transactions	Expected result
1	T1 500, T2 1000, T1 500	Duplicate: T1; no high transaction (1000 is not above the limit)
2	T3 200, T4 300	No duplicates; no high transaction
3	T5 1500, T5 1500	Duplicate: T5; T5 1500 reported as high transaction
4	T6 700, T7 2000, T7 2000	Duplicate: T7; T7 2000 reported as high transaction
5	T8 100, T9 2500, T10 400	No duplicates; high transaction: T9 2500

9. Output Screenshot

```
PS C:\Users\Administrator\Desktop\python>  & 'C:\Users\Administrator\AppData\Local\Python\pythoncore-3.14-64\python.exe'
'c:\Users\Administrator\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55824' '-
-' 'C:\Users\Administrator\Desktop\python\case2.py'

Transactions: [('T1', 500), ('T2', 1000), ('T1', 500)]
Duplicates: ['T1']

Transactions: [('T3', 200), ('T4', 300)]
Duplicates: []

Transactions: [('T5', 1500), ('T5', 1500)]
Duplicates: ['T5']
High transaction: T5 1500
High transaction: T5 1500

Transactions: [('T6', 700), ('T7', 2000), ('T7', 2000)]
Duplicates: ['T7']
High transaction: T7 2000
High transaction: T7 2000

Transactions: [('T8', 100), ('T9', 2500), ('T10', 400)]
Duplicates: []
High transaction: T9 2500
PS C:\Users\Administrator\Desktop\python>
```

10. Conclusion

This program detects repeated transaction IDs and flags high-value payments in each group of transactions. It helps in spotting possible double entries and large transactions that need review.

CASE 03: Password Policy Validation

1. Problem Statement

Weak passwords make user accounts easy to attack. A password policy defines the minimum strength that a password must have before it is accepted. This project develops a Python program that checks a list of passwords against a policy covering length, uppercase letters, lowercase letters, digits and special characters, and reports whether each password is valid or invalid.

2. Objectives

1. To develop a Python program that validates passwords against a security policy.
2. To check the minimum password length of 8 characters.
3. To verify the presence of uppercase letters, lowercase letters, digits and special characters.
4. To classify each character of a password using string methods.
5. To display a clear Valid or Invalid message for every password.

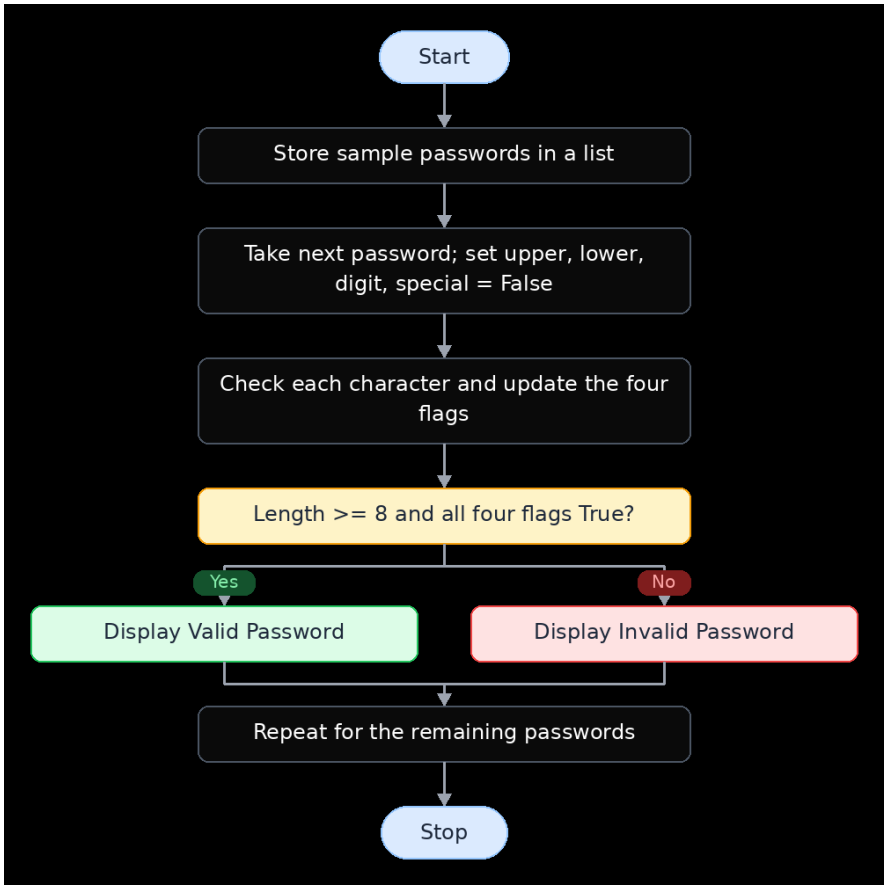
3. Assumptions

1. The minimum password length is 8 characters.
2. A valid password must contain at least one uppercase letter (A-Z).
3. A valid password must contain at least one lowercase letter (a-z).
4. A valid password must contain at least one digit (0-9).
5. Any character that is not a letter or a digit is treated as a special character.
6. A password is accepted only when all the rules are satisfied.

4. Algorithm

1. Start the program.
2. Store the sample passwords in a list.
3. For each password, set the flags upper, lower, digit and special to False.
4. Examine every character: set upper for an uppercase letter, lower for a lowercase letter, digit for a number, and special for any other character.
5. Check that the password length is at least 8 and that all four flags are True.
6. If all conditions are satisfied, display Valid Password.
7. Otherwise, display Invalid Password.
8. Repeat for all passwords.
9. Stop the program.

5. Flowchart



6. Pseudocode

```
START

SET passwords = list of sample passwords

FOR each password in passwords:
    upper = False
    lower = False
    digit = False
    special = False

    FOR each character ch in password:
        IF ch is uppercase:
            upper = True
        ELSE IF ch is lowercase:
            lower = True
        ELSE IF ch is a digit:
            digit = True
        ELSE:
            special = True

    IF length >= 8 AND upper AND lower AND digit AND special:
        DISPLAY password, "- Valid Password"
    ELSE:
        DISPLAY password, "- Invalid Password"

STOP
```

7. Python Source Code / Program

```
passwords = [
    "Python@123",
    "hello",
    "College123",
    "Abc@4567",
    "Student#456"
]

for password in passwords:
    upper = False
    lower = False
    digit = False
    special = False

    for ch in password:
        if ch.isupper():
            upper = True
        elif ch.islower():
            lower = True
        elif ch.isdigit():
            digit = True
        else:
            special = True

    if len(password) >= 8 and upper and lower and digit and special:
        print(password, "- Valid Password")
    else:
        print(password, "- Invalid Password")
```

8. Test Cases

Test	Input password	Expected result
1	Python@123	Valid
2	hello	Invalid - too short; digit, uppercase and special character missing
3	College123	Invalid - special character missing
4	Abc@4567	Valid
5	Student#456	Valid

9. Output Screenshot

```
PS C:\Users\Administrator\Desktop\python>  & 'C:\Users\Administrator\AppData\Local\Python\pythoncore-3.14-64\python.exe'
'c:\Users\Administrator\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '63056' '-
-' 'C:\Users\Administrator\Desktop\python\case3.py'
Python@123 - Valid Password
hello - Invalid Password
College123 - Invalid Password
Abc@4567 - Valid Password
Student#456 - Valid Password
PS C:\Users\Administrator\Desktop\python>
```

10. Conclusion

This program checks each password against the defined policy and reports whether it is valid. It shows how a few simple flags can be combined to enforce several security rules in one pass.

CASE 04: Student Email and Identifier Validation

1. Problem Statement

Student records are useful only when the email addresses and roll numbers stored in them are correctly formatted. Wrongly formatted entries cause delivery failures and identification errors. This project develops a Python program that validates an email address and a roll number for each student and displays whether each one is valid or invalid.

2. Objectives

1. To develop a Python program that validates student email addresses.
2. To validate roll numbers against a fixed format.
3. To use string methods such as index, rfind, isdigit and isalpha.
4. To use slicing to examine the parts of a roll number.
5. To display the validation result for the email and the roll number of every student.

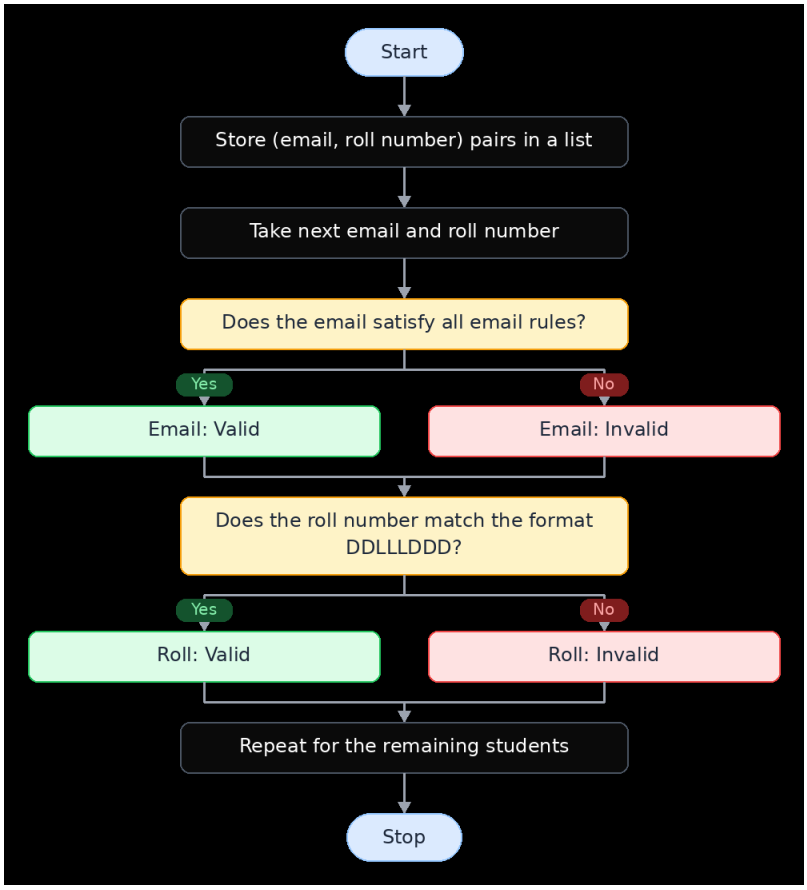
3. Assumptions

1. An email is valid only if it contains the symbols @ and . .
2. The @ symbol must not be the first character of the email.
3. The last dot must appear after the @ symbol with at least one character between them.
4. The last dot must not be the last character of the email.
5. A roll number must be exactly 8 characters long.
6. The format of a roll number is two digits, three letters and three digits, for example 22CSE101.

4. Algorithm

1. Start the program.
2. Store the email and roll number pairs of the students in a list.
3. Take the next email and roll number.
4. Check that the email contains @ and . , that @ is not the first character, that the last dot comes after @ with a gap, and that the last dot is not at the end.
5. Check that the roll number has 8 characters, the first two are digits, the next three are letters and the last three are digits.
6. Display the email and whether it is Valid or Invalid.
7. Display the roll number and whether it is Valid or Invalid.
8. Repeat for all students.
9. Stop the program.

5. Flowchart



6. Pseudocode

```
START

SET students = list of (email, roll)

FOR each (email, roll) in students:
    email_valid = "@" in email AND "." in email
                  AND index of "@" > 0
                  AND last "." is after "@" with a gap
                  AND last "." is not the last character

    roll_valid = length of roll = 8
                 AND first 2 characters are digits
                 AND next 3 characters are letters
                 AND last 3 characters are digits

    DISPLAY email
    IF email_valid: DISPLAY "Valid" ELSE DISPLAY "Invalid"
    DISPLAY roll
    IF roll_valid: DISPLAY "Valid" ELSE DISPLAY "Invalid"

STOP
```

7. Python Source Code / Program

```
students = [
    ("abc@gmail.com", "22CSE101"),
    ("xyz@yahoo.com", "23ECE102"),
    ("studentgmail.com", "22CSE103"),
    ("hello@gmail.com", "2ACSE101"),
    ("test@gmail.com", "24ITC105")
]

for email, roll in students:
    email_valid = (
        "@" in email and "." in email
        and email.index("@") > 0
        and email.rfind(".") > email.index("@") + 1
        and email.rfind(".") < len(email) - 1
    )

    roll_valid = (
        len(roll) == 8
        and roll[:2].isdigit()
        and roll[2:5].isalpha()
        and roll[5:].isdigit()
    )

    print("\nEmail:", email)
    print("Email:", "Valid" if email_valid else "Invalid")
    print("Roll:", roll)
    print("Roll:", "Valid" if roll_valid else "Invalid")
```

8. Test Cases

Test	Email / Roll number	Expected result
1	abc@gmail.com / 22CSE101	Email Valid; Roll Valid
2	xyz@yahoo.com / 23ECE102	Email Valid; Roll Valid
3	studentgmail.com / 22CSE103	Email Invalid (no @ symbol); Roll Valid
4	hello@gmail.com / 2ACSE101	Email Valid; Roll Invalid (first two characters are not digits)
5	test@gmail.com / 24ITC105	Email Valid; Roll Valid

9. Output Screenshot

```
PS C:\Users\Administrator\Desktop\python>  & 'C:\Users\Administrator\AppData\Local\Python\pythoncore-3.14-64\python.exe'
'c:\Users\Administrator\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '61065' '-
-' 'C:\Users\Administrator\Desktop\python\case4.py'

Email: abc@gmail.com
Email: Valid
Roll: 22CSE101
Roll: Valid

Email: xyz@yahoo.com
Email: Valid
Roll: 23ECE102
Roll: Valid

Email: studentgmail.com
Email: Invalid
Roll: 22CSE103
Roll: Valid

Email: hello@gmail.com
Email: Valid
Roll: 2ACSE101
Roll: Invalid

Email: test@gmail.com
Email: Valid
Roll: 24ITC105
Roll: Valid
PS C:\Users\Administrator\Desktop\python>
```

10. Conclusion

This program validates email addresses and roll numbers using string methods and slicing. It shows how a data format can be checked reliably before the data is stored or used.

CASE 05: Student Marks Matrix Analysis

1. Problem Statement

Teachers often need to analyse marks of several students across multiple subjects. Calculating totals, averages and the top performer by hand is slow and error-prone. This project develops a Python program that stores marks in a matrix, calculates the total and average for every student, gives a result remark and identifies the top student.

2. Objectives

1. To store the marks of several students in a two-dimensional list (matrix).
2. To calculate the total and average marks of each student.
3. To classify each student as Good or Needs Improvement.
4. To identify the student with the highest total marks.
5. To display the analysis for every student clearly.

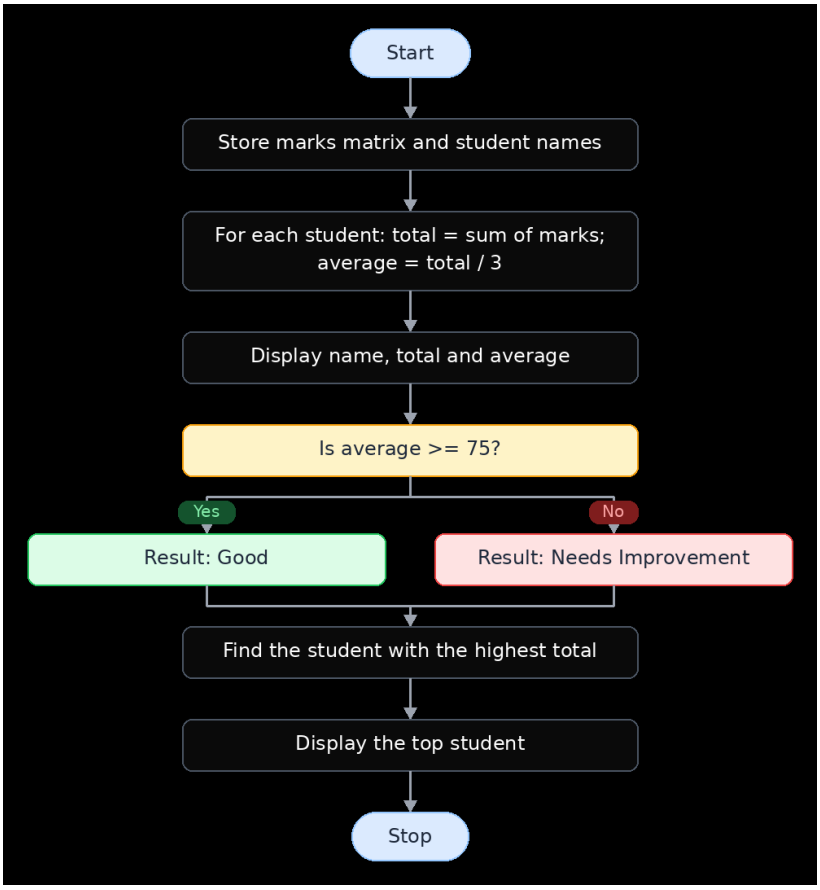
3. Assumptions

1. Each student has marks in three subjects.
2. There are five students and their names are stored in a separate list in the same order as the marks.
3. The average is calculated as total marks divided by 3.
4. A student with an average of 75 or more gets the result Good.
5. A student with an average below 75 gets the result Needs Improvement.
6. The top student is the one with the highest total; for equal totals the first student found is kept.

4. Algorithm

1. Start the program.
2. Store the marks matrix and the list of student names.
3. For each student, calculate the total using sum and the average by dividing the total by 3.
4. Display the name, total and average.
5. If the average is at least 75, display Good; otherwise display Needs Improvement.
6. Set the top student index to the first student.
7. Compare the total of every other student with that of the current top student and update the index when a higher total is found.
8. Display the name of the top student.
9. Stop the program.

5. Flowchart



6. Pseudocode

```
START

SET students = matrix of marks (5 students x 3 subjects)
SET names = ["Anu", "Balu", "Charan", "Divya", "Esha"]

FOR i = 0 TO 4:
    total = SUM of students[i]
    average = total / 3

    DISPLAY names[i], total, average

    IF average >= 75:
        DISPLAY "Result: Good"
    ELSE:
        DISPLAY "Result: Needs Improvement"

top = 0
FOR i = 1 TO 4:
    IF SUM(students[i]) > SUM(students[top]):
        top = i

DISPLAY "Top Student:", names[top]

STOP
```

7. Python Source Code / Program

```
students = [
    [85, 90, 80],
    [70, 65, 75],
    [95, 92, 98],
    [60, 55, 65],
    [80, 85, 90]
]

names = ["Anu", "Balu", "Charan", "Divya", "Esha"]

for i in range(5):
    total = sum(students[i])
    average = total / 3

    print("\nStudent:", names[i])
    print("Total:", total)
    print("Average:", average)

    if average >= 75:
        print("Result: Good")
    else:
        print("Result: Needs Improvement")

top = 0

for i in range(1, 5):
    if sum(students[i]) > sum(students[top]):
        top = i

print("\nTop Student:", names[top])
```

8. Test Cases

Test	Student (marks)	Expected result
1	Anu (85, 90, 80)	Total 255; Average 85.0; Good
2	Balu (70, 65, 75)	Total 210; Average 70.0; Needs Improvement
3	Charan (95, 92, 98)	Total 285; Average 95.0; Good
4	Divya (60, 55, 65)	Total 180; Average 60.0; Needs Improvement
5	Esha (80, 85, 90)	Total 255; Average 85.0; Good
6	All five students	Top Student: Charan

9. Output Screenshot

```
PS C:\Users\Administrator\Desktop\python>  & 'C:\Users\Administrator\AppData\Local\Python\pythoncore-3.14-64\python.exe'
'c:\Users\Administrator\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '50404' '-
-' 'C:\Users\Administrator\Desktop\python\case5.py'

Student: Anu
Total: 255
Average: 85.0
Result: Good

Student: Balu
Total: 210
Average: 70.0
Result: Needs Improvement

Student: Charan
Total: 285
Average: 95.0
Result: Good

Student: Divya
Total: 180
Average: 60.0
Result: Needs Improvement

Student: Esha
Total: 255
Average: 85.0
Result: Good

Top Student: Charan
PS C:\Users\Administrator\Desktop\python>
```

10. Conclusion

This program analyses a marks matrix to calculate totals and averages, give a result for each student and find the top performer. It shows how two-dimensional lists and simple comparisons support quick academic analysis.

CASE 06: Weekly Electricity Consumption Matrix

1. Problem Statement

Households and institutions need to track how much electricity different appliances use each day so that high consumption can be noticed early. This project develops a Python program that stores the daily units used by a fan, a light and a TV for five weekdays in a matrix, calculates the total units for each day and classifies the day as normal or high consumption.

2. Objectives

1. To store the daily consumption of several appliances in a matrix.
2. To display the units used by each appliance for every day.
3. To calculate the total units consumed on each day.
4. To classify each day as High Consumption or Normal Consumption.
5. To present the weekly report in a clear, readable format.

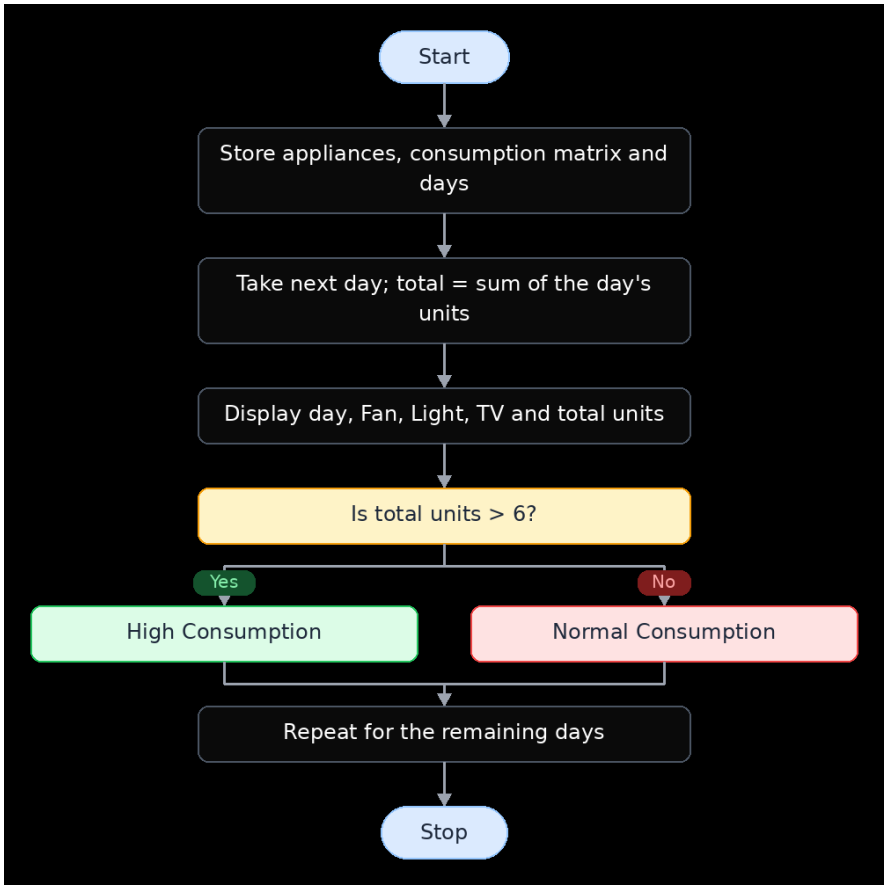
3. Assumptions

1. The appliances considered are Fan, Light and TV.
2. Consumption is recorded for five days from Monday to Friday.
3. Each row of the matrix holds the units used by Fan, Light and TV in that order.
4. The total units for a day is the sum of the three appliance values.
5. A day with total units greater than 6 is High Consumption.
6. A day with total units of 6 or less is Normal Consumption.

4. Algorithm

1. Start the program.
2. Store the appliance names, the consumption matrix and the names of the days.
3. For each day, calculate the total units using sum.
4. Display the day, the units used by Fan, Light and TV, and the total units.
5. If the total is greater than 6, display High Consumption.
6. Otherwise, display Normal Consumption.
7. Repeat for all five days.
8. Stop the program.

5. Flowchart



6. Pseudocode

```
START

SET appliances = ["Fan", "Light", "TV"]
SET test_cases = matrix of daily units (5 days x 3 appliances)
SET days = ["Monday", ..., "Friday"]

FOR i = 0 TO 4:
    total = SUM of test_cases[i]

    DISPLAY days[i]
    DISPLAY Fan, Light and TV units
    DISPLAY total

    IF total > 6:
        DISPLAY "High Consumption"
    ELSE:
        DISPLAY "Normal Consumption"

STOP
```

7. Python Source Code / Program

```
appliances = ["Fan", "Light", "TV"]

test_cases = [
    [2, 1, 3],
    [3, 2, 4],
    [1, 1, 2],
    [4, 2, 3],
    [2, 3, 5]
]

days = ["Monday", "Tuesday", "Wednesday",
        "Thursday", "Friday"]

for i in range(5):
    total = sum(test_cases[i])

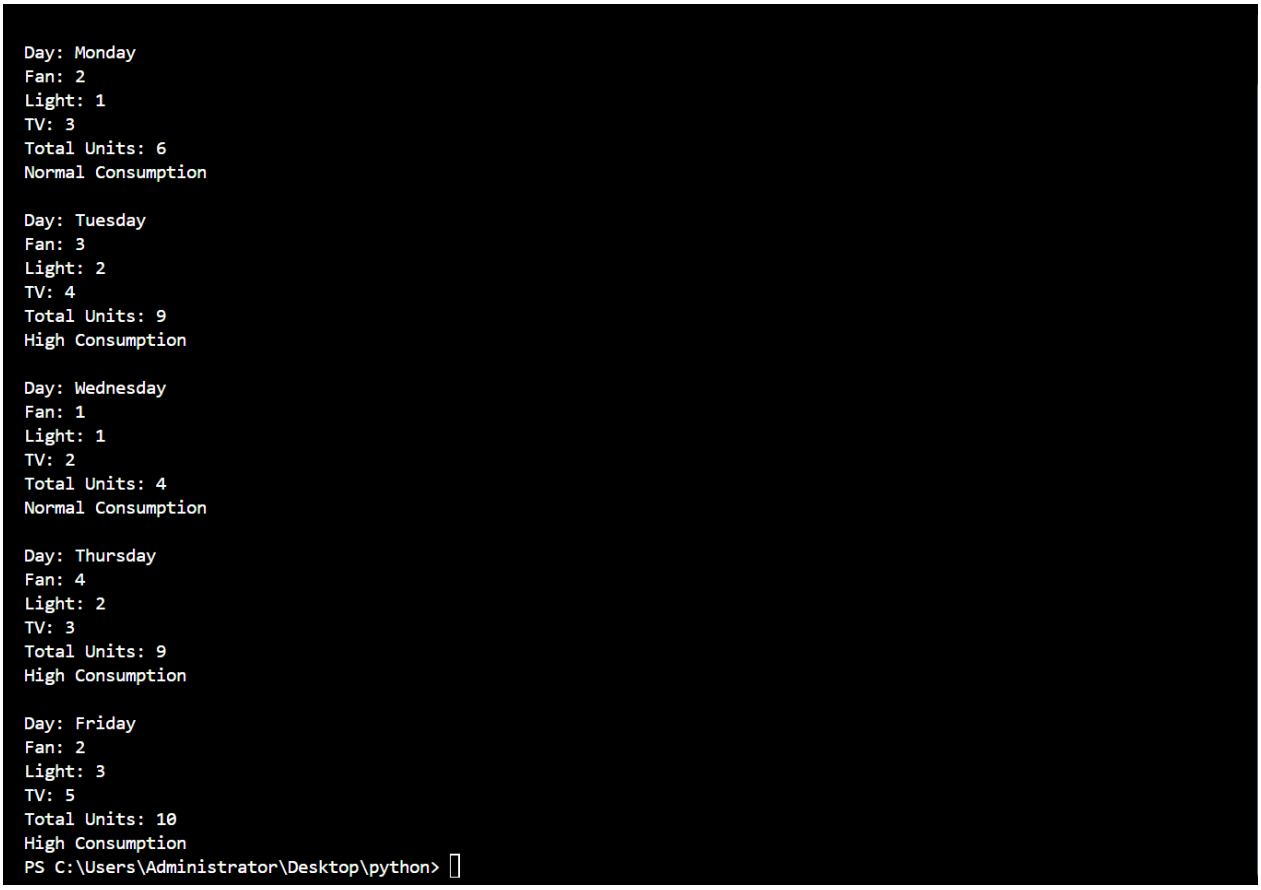
    print("\nDay:", days[i])
    print("Fan:", test_cases[i][0])
    print("Light:", test_cases[i][1])
    print("TV:", test_cases[i][2])
    print("Total Units:", total)

    if total > 6:
        print("High Consumption")
    else:
        print("Normal Consumption")
```

8. Test Cases

Test	Day (Fan, Light, TV)	Expected result
1	Monday (2, 1, 3)	Total 6; Normal Consumption
2	Tuesday (3, 2, 4)	Total 9; High Consumption
3	Wednesday (1, 1, 2)	Total 4; Normal Consumption
4	Thursday (4, 2, 3)	Total 9; High Consumption
5	Friday (2, 3, 5)	Total 10; High Consumption

9. Output Screenshot



10. Conclusion

This program summarises weekly appliance usage, calculates daily totals and flags days of high consumption. It shows how matrices and conditions can support simple energy monitoring.

CASE 07: Employee Shift Conflict Detection

1. Problem Statement

When an employee is assigned more than one shift in a day, the shifts must not overlap. Overlapping shifts cause scheduling errors and unfair workloads. This project develops a Python program that checks the shifts of each employee and reports a shift conflict whenever two shifts of the same employee overlap in time.

2. Objectives

1. To develop a Python program that detects overlapping shifts for employees.
2. To compare every pair of shifts in a group of shifts.
3. To check that the shifts belong to the same employee.
4. To apply the time-overlap condition correctly.
5. To display the conflicting employee or report that there is no conflict.

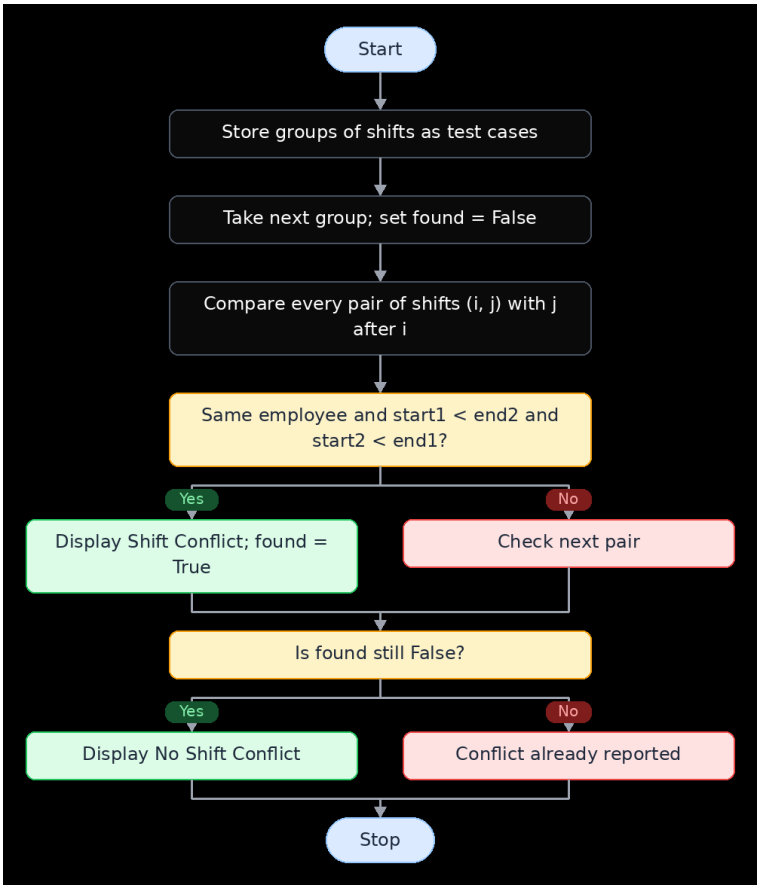
3. Assumptions

1. Each shift is stored as the employee name, start hour and end hour.
2. Times are given in 24-hour format as whole hours.
3. Two shifts conflict if they belong to the same employee and their time ranges overlap.
4. Two shifts overlap when start1 < end2 and start2 < end1.
5. Shifts that only touch at an end time, such as 9 to 11 and 11 to 15, do not conflict.
6. Each test case contains two shifts.

4. Algorithm

1. Start the program.
2. Store the groups of shifts in a list of test cases.
3. For each group, display the shifts and set found to False.
4. Compare every shift i with every later shift j in the group.
5. If both shifts belong to the same employee and start1 < end2 and start2 < end1, display the shift conflict and set found to True.
6. After all comparisons, if found is still False, display No Shift Conflict.
7. Repeat for all groups.
8. Stop the program.

5. Flowchart



6. Pseudocode

```
START

SET test_cases = groups of (name, start, end)

FOR each group (shifts) in test_cases:
    DISPLAY shifts
    found = False

    FOR i = 0 TO length(shifts) - 1:
        FOR j = i + 1 TO length(shifts) - 1:
            (name1, start1, end1) = shifts[i]
            (name2, start2, end2) = shifts[j]

            IF name1 = name2 AND start1 < end2 AND start2 < end1:
                DISPLAY name1, "- Shift Conflict"
                found = True

    IF found is False:
        DISPLAY "No Shift Conflict"

STOP
```

7. Python Source Code / Program

```
test_cases = [
    [("Ravi", 9, 13), ("Ravi", 12, 16)],
    [("Anu", 9, 12), ("Anu", 13, 17)],
    [("Balu", 8, 12), ("Balu", 10, 14)],
    [("Kiran", 9, 11), ("Kiran", 11, 15)],
    [("Divya", 10, 14), ("Divya", 12, 16)]
]

for shifts in test_cases:
    print("\nShifts:", shifts)
    found = False

    for i in range(len(shifts)):
        for j in range(i+1, len(shifts)):
            name1, start1, end1 = shifts[i]
            name2, start2, end2 = shifts[j]

            if name1 == name2 and start1 < end2 and start2 < end1:
                print(name1, "- Shift Conflict")
                found = True

    if not found:
        print("No Shift Conflict")
```

8. Test Cases

Test	Shifts	Expected result
1	Ravi 9-13, Ravi 12-16	Ravi - Shift Conflict (12 to 13 overlaps)
2	Anu 9-12, Anu 13-17	No Shift Conflict
3	Balu 8-12, Balu 10-14	Balu - Shift Conflict (10 to 12 overlaps)
4	Kiran 9-11, Kiran 11-15	No Shift Conflict (shifts only touch at 11)
5	Divya 10-14, Divya 12-16	Divya - Shift Conflict (12 to 14 overlaps)

9. Output Screenshot

```
PS C:\Users\Administrator\Desktop\python>  & 'C:\Users\Administrator\AppData\Local\Python\pythoncore-3.14-64\python.exe'
'c:\Users\Administrator\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '50564' '-
-' 'C:\Users\Administrator\Desktop\python\case7.py'

Shifts: [('Ravi', 9, 13), ('Ravi', 12, 16)]
Ravi - Shift Conflict

Shifts: [('Anu', 9, 12), ('Anu', 13, 17)]
No Shift Conflict

Shifts: [('Balu', 8, 12), ('Balu', 10, 14)]
Balu - Shift Conflict

Shifts: [('Kiran', 9, 11), ('Kiran', 11, 15)]
No Shift Conflict

Shifts: [('Divya', 10, 14), ('Divya', 12, 16)]
Divya - Shift Conflict
PS C:\Users\Administrator\Desktop\python>
```

10. Conclusion

This program compares pairs of shifts and reports an employee's conflict whenever two time ranges overlap. It shows how a simple interval condition can detect scheduling clashes reliably.

CASE 08: Product Catalogue Audit

1. Problem Statement

A store must regularly audit its product catalogue to find invalid entries and products that need restocking. Entries with a non-positive price or a negative stock value are errors, and products with low stock must be replenished. This project develops a Python program that audits a list of products, calculates the stock value of valid products and reports whether replenishment is required.

2. Objectives

1. To develop a Python program that audits product records.
2. To identify invalid products with a wrong price or stock value.
3. To calculate the total stock value of valid products.
4. To identify products whose stock is below the replenishment level.
5. To display a clear audit report for every product.

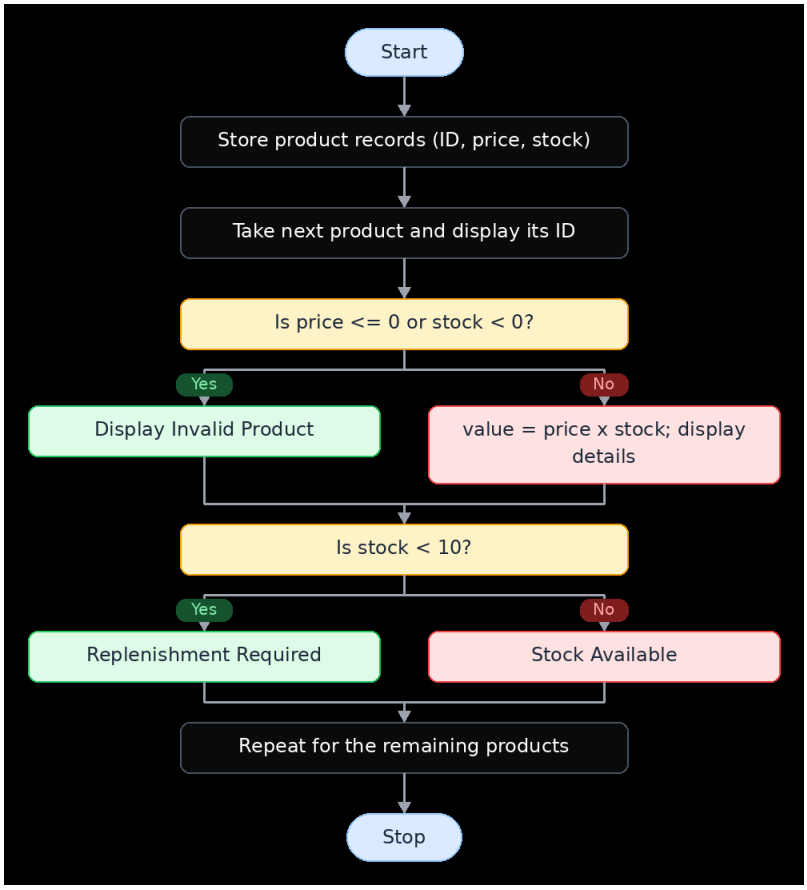
3. Assumptions

1. Each product is stored as a product ID, a price and a stock quantity.
2. A product is invalid if its price is less than or equal to 0 or its stock is less than 0.
3. The total value of a valid product is price multiplied by stock.
4. Replenishment is required when the stock is less than 10.
5. If the stock is 10 or more, the stock is considered available.
6. A product with zero stock is valid but needs replenishment.

4. Algorithm

1. Start the program.
2. Store the products as a list of (product ID, price, stock).
3. Take the next product and display its ID.
4. If the price is 0 or less, or the stock is negative, display Invalid Product.
5. Otherwise, calculate the total value as price multiplied by stock.
6. Display the price, stock and total value.
7. If the stock is less than 10, display Replenishment Required; otherwise display Stock Available.
8. Repeat for all products.
9. Stop the program.

5. Flowchart



6. Pseudocode

```
START

SET test_cases = list of (pid, price, stock)

FOR each (pid, price, stock) in test_cases:
    DISPLAY "Product:", pid

    IF price <= 0 OR stock < 0:
        DISPLAY "Invalid Product"
    ELSE:
        value = price * stock
        DISPLAY price, stock, value

        IF stock < 10:
            DISPLAY "Replenishment Required"
        ELSE:
            DISPLAY "Stock Available"

STOP
```

7. Python Source Code / Program

```
test_cases = [
    ("P101", 100, 20),
    ("P102", 250, 5),
    ("P103", -50, 10),
    ("P104", 500, 0),
    ("P105", 300, 15)
]

for pid, price, stock in test_cases:
    print("\nProduct:", pid)

    if price <= 0 or stock < 0:
        print("Invalid Product")
    else:
        value = price * stock
        print("Price:", price)
        print("Stock:", stock)
        print("Total Value:", value)

        if stock < 10:
            print("Replenishment Required")
        else:
            print("Stock Available")
```

8. Test Cases

Test	Product (price, stock)	Expected result
1	P101 (100, 20)	Total Value 2000; Stock Available
2	P102 (250, 5)	Total Value 1250; Replenishment Required
3	P103 (-50, 10)	Invalid Product (negative price)
4	P104 (500, 0)	Total Value 0; Replenishment Required
5	P105 (300, 15)	Total Value 4500; Stock Available

9. Output Screenshot

```
PS C:\Users\Administrator\Desktop\python>  & 'C:\Users\Administrator\AppData\Local\Python\pythoncore-3.14-64\python.exe'
'c:\Users\Administrator\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55500' '-
-' 'C:\Users\Administrator\Desktop\python\case8.py'

Product: P101
Price: 100
Stock: 20
Total Value: 2000
Stock Available

Product: P102
Price: 250
Stock: 5
Total Value: 1250
Replenishment Required

Product: P103
Invalid Product

Product: P104
Price: 500
Stock: 0
Total Value: 0
Replenishment Required

Product: P105
Price: 300
Stock: 15
Total Value: 4500
Stock Available
PS C:\Users\Administrator\Desktop\python>
```

10. Conclusion

This program audits product records, rejects invalid entries, calculates stock value and flags products that need replenishment. It helps keep a product catalogue accurate and the stock well managed.

CASE 09: Environmental Monitoring Data Analyzer

1. Problem Statement

Environmental monitoring stations record repeated readings, such as pollution levels, at different locations. Reviewing these readings manually makes it hard to see which areas are above the safe limit. This project develops a Python program that analyses the readings of several areas, calculates the average, maximum and minimum values and reports whether each area is within the safe limit.

2. Objectives

1. To develop a Python program that analyses environmental readings for several locations.
2. To calculate the average reading for each location.
3. To find the maximum and minimum readings using built-in functions.
4. To compare the average with the safe limit.
5. To display the status of every location clearly.

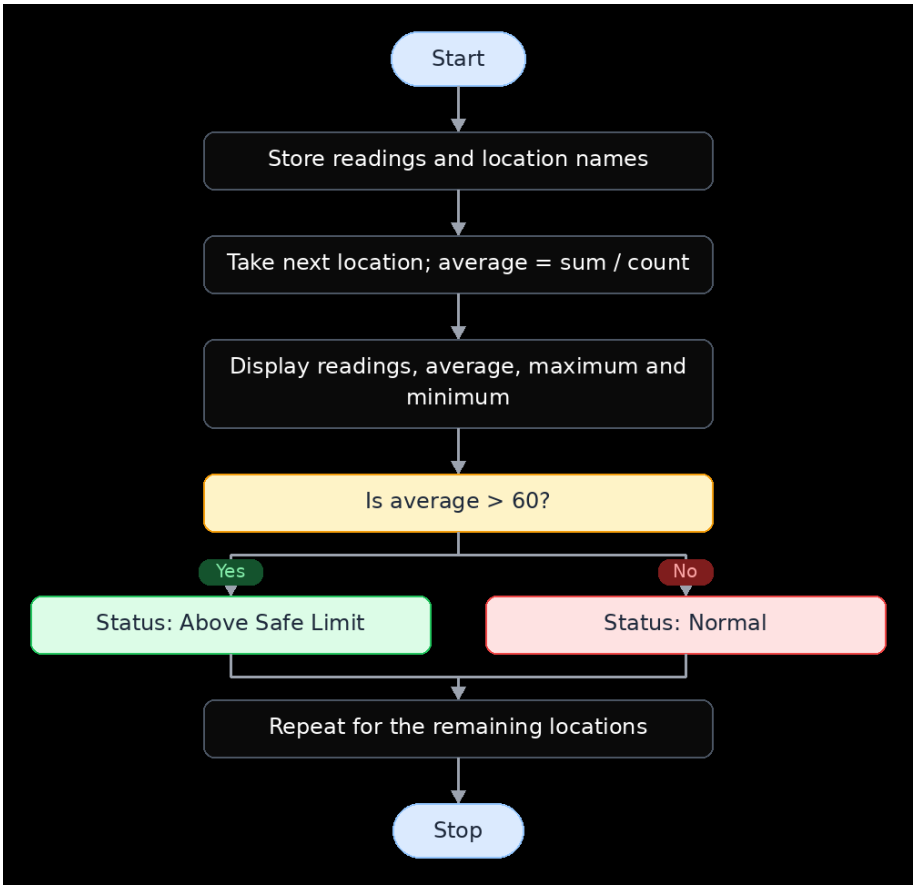
3. Assumptions

1. Each location has three readings.
2. Readings are stored in a list, one inner list per location, in the same order as the location names.
3. The locations considered are Area A to Area E.
4. The average is the sum of the readings divided by the number of readings.
5. The safe limit for the average reading is 60.
6. An average greater than 60 is reported as Above Safe Limit; otherwise the status is Normal.

4. Algorithm

1. Start the program.
2. Store the readings of the five areas and the list of location names.
3. For each location, take its list of readings.
4. Calculate the average as sum of readings divided by the number of readings.
5. Display the location, readings, average, maximum and minimum.
6. If the average is greater than 60, display Status: Above Safe Limit.
7. Otherwise, display Status: Normal.
8. Repeat for all locations.
9. Stop the program.

5. Flowchart



6. Pseudocode

```
START

SET test_cases = readings of 5 areas (3 readings each)
SET locations = ["Area A", ..., "Area E"]

FOR i = 0 TO 4:
    readings = test_cases[i]
    average = SUM(readings) / LENGTH(readings)

    DISPLAY locations[i]
    DISPLAY readings
    DISPLAY average
    DISPLAY MAX(readings)
    DISPLAY MIN(readings)

    IF average > 60:
        DISPLAY "Status: Above Safe Limit"
    ELSE:
        DISPLAY "Status: Normal"

STOP
```

7. Python Source Code / Program

```
test_cases = [
    [35, 40, 45],
    [80, 85, 90],
    [25, 30, 28],
    [55, 60, 65],
    [90, 95, 100]
]

locations = ["Area A", "Area B", "Area C",
             "Area D", "Area E"]

for i in range(5):
    readings = test_cases[i]
    average = sum(readings) / len(readings)

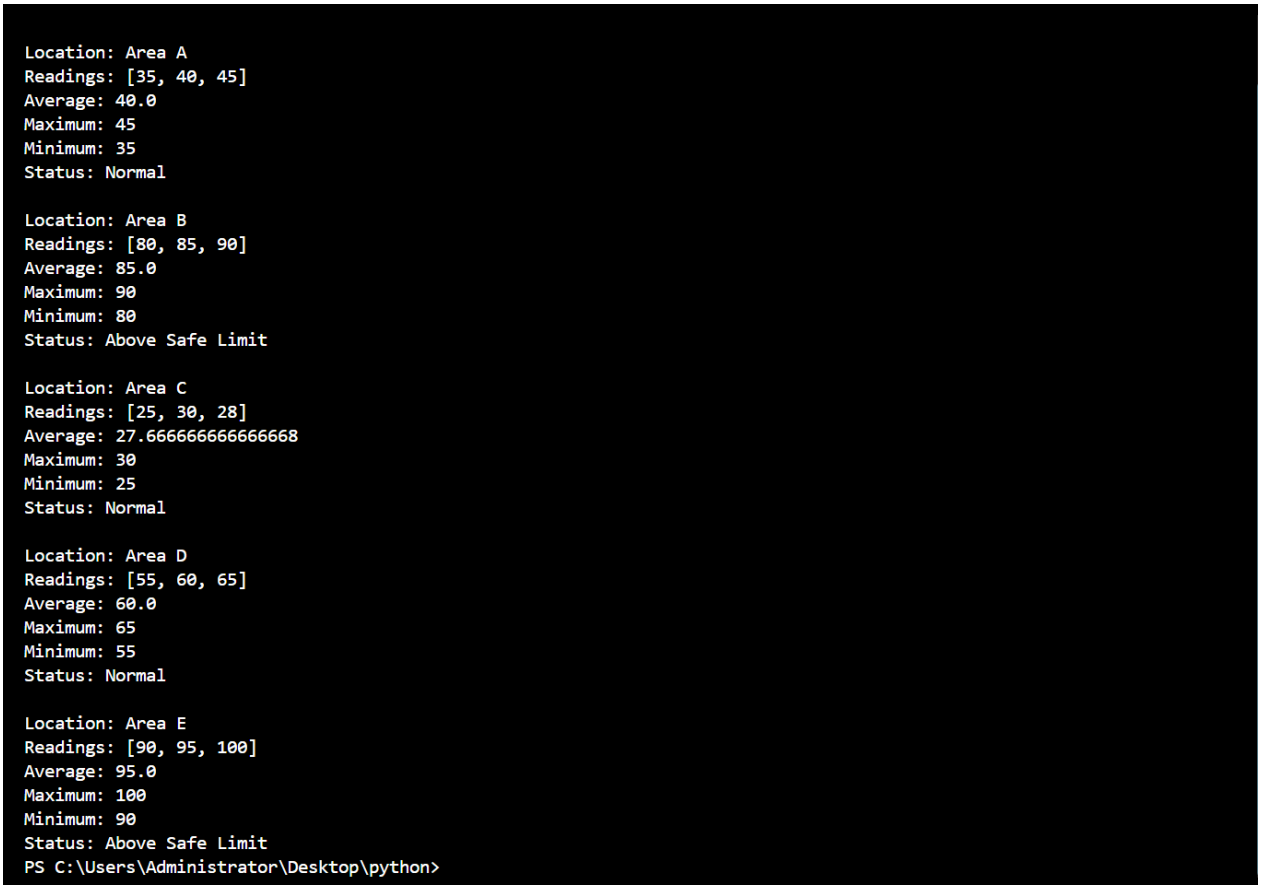
    print("\nLocation:", locations[i])
    print("Readings:", readings)
    print("Average:", average)
    print("Maximum:", max(readings))
    print("Minimum:", min(readings))

    if average > 60:
        print("Status: Above Safe Limit")
    else:
        print("Status: Normal")
```

8. Test Cases

Test	Location (readings)	Expected result
1	Area A (35, 40, 45)	Average 40.0; Max 45; Min 35; Normal
2	Area B (80, 85, 90)	Average 85.0; Max 90; Min 80; Above Safe Limit
3	Area C (25, 30, 28)	Average 27.67 (approx.); Max 30; Min 25; Normal
4	Area D (55, 60, 65)	Average 60.0; Max 65; Min 55; Normal (60 is not above the limit)
5	Area E (90, 95, 100)	Average 95.0; Max 100; Min 90; Above Safe Limit

9. Output Screenshot



10. Conclusion

This program summarises environmental readings by finding the average, maximum and minimum for each area and compares the average with a safe limit. It shows how simple statistics can highlight areas that need attention.

CASE 10: Advanced String-Based Registration Record Analyzer

1. Problem Statement

University registration records follow a fixed text format such as 22-CSE-101, which holds the year, the branch and the student number. Records in the wrong format or repeated records cause errors in the database. This project develops a Python program that splits each registration string, validates its format, detects duplicate records and displays the year, branch and student number of valid records.

2. Objectives

1. To develop a Python program that analyses registration records stored as strings.
2. To split each record into year, branch and student number using the hyphen.
3. To validate the format of every part of the record.
4. To detect records that have already been entered.
5. To display the year, branch and student number of every valid record.

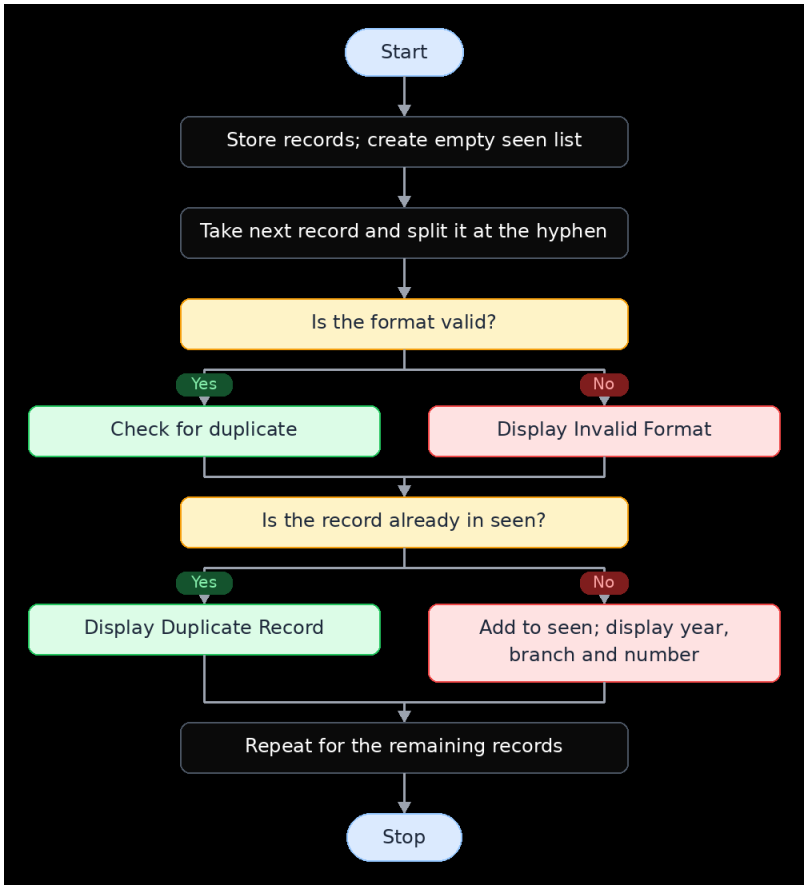
3. Assumptions

1. A record has exactly three parts separated by hyphens: year, branch and student number.
2. The year must have exactly 2 characters and all of them must be digits.
3. The branch must contain letters only and have at least 2 characters.
4. The student number must have exactly 3 characters and all of them must be digits.
5. A valid record that has already appeared earlier in the list is a duplicate.
6. Records that fail the format check are reported as Invalid Format.

4. Algorithm

1. Start the program.
2. Store the registration records in a list and create an empty seen list.
3. Take the next record and split it at the hyphen into parts.
4. Check that there are 3 parts, that the year has 2 digits, that the branch is alphabetic with at least 2 letters and that the student number has 3 digits.
5. Display the registration record.
6. If the format is not valid, display Invalid Format.
7. Otherwise, if the record is already in the seen list, display Duplicate Record.
8. Otherwise, add it to the seen list and display Valid Record with the year, branch and student number.
9. Repeat for all records.
10. Stop the program.

5. Flowchart



6. Pseudocode

```
START

SET records = list of registration strings
SET seen = empty list

FOR each record in records:
    parts = SPLIT record at "-"

    valid = LENGTH(parts) = 3
            AND LENGTH(parts[0]) = 2 AND parts[0] is digits
            AND parts[1] is letters AND LENGTH(parts[1]) >= 2
            AND LENGTH(parts[2]) = 3 AND parts[2] is digits

    DISPLAY "Registration:", record

    IF NOT valid:
        DISPLAY "Invalid Format"
    ELSE IF record is in seen:
        DISPLAY "Duplicate Record"
    ELSE:
        ADD record to seen
        DISPLAY "Valid Record"
        DISPLAY Year, Branch, Student Number

STOP
```

7. Python Source Code / Program

```
records = [
    "22-CSE-101",
    "23-ECE-205",
    "2A-CSE-110",
    "24-MECH-305",
    "22-CSE-101"
]
seen = []
for record in records:
    parts = record.split("-")
    valid = (
        len(parts) == 3
        and len(parts[0]) == 2
        and parts[0].isdigit()
        and parts[1].isalpha()
        and len(parts[1]) >= 2
        and len(parts[2]) == 3
        and parts[2].isdigit()
    )
    print("\nRegistration:", record)
    if not valid:
        print("Invalid Format")
    elif record in seen:
        print("Duplicate Record")
    else:
        seen.append(record)
        print("Valid Record")
        print("Year:", parts[0])
        print("Branch:", parts[1])
        print("Student Number:", parts[2])
```

8. Test Cases

Test	Registration record	Expected result
1	22-CSE-101	Valid Record; Year 22, Branch CSE, Student Number 101
2	23-ECE-205	Valid Record; Year 23, Branch ECE, Student Number 205
3	2A-CSE-110	Invalid Format (year is not two digits)
4	24-MECH-305	Valid Record; Year 24, Branch MECH, Student Number 305
5	22-CSE-101 (entered again)	Duplicate Record

9. Output Screenshot

```
PS C:\Users\Administrator\Desktop\python>  & 'C:\Users\Administrator\AppData\Local\Python\pythoncore-3.14-64\python.exe'
'c:\Users\Administrator\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55890' '-
-' 'C:\Users\Administrator\Desktop\python\cas10.py'

Registration: 22-CSE-101
Valid Record
Year: 22
Branch: CSE
Student Number: 101

Registration: 23-ECE-205
Valid Record
Year: 23
Branch: ECE
Student Number: 205

Registration: 2A-CSE-110
Invalid Format

Registration: 24-MECH-305
Valid Record
Year: 24
Branch: MECH
Student Number: 305

Registration: 22-CSE-101
Duplicate Record
PS C:\Users\Administrator\Desktop\python>
```

10. Conclusion

This program splits registration strings, validates each part, detects duplicate records and displays the details of valid ones. It shows how string handling and list checks can keep record data clean and consistent.